ECE661: Homework 1

Fall 2026

Due Date: Wednesday, September 2, 2026, 11:59 pm ET

TA: Somosmita Mitra (mitra26@purdue.edu)

Turn in typed solutions via Gradescope. Post questions to Piazza. Additional instructions can be found at the end. Late submissions will be accepted with penalty: -10 points per late day, up to 5 days.

1 Introduction

Everything in this homework comes from Lecture 2. The line through two points is $\mathbf{l} = \mathbf{x}_1 \times \mathbf{x}_2$, the point where two lines meet is $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$, a conic is $\mathbf{x}^T\mathbf{C}\mathbf{x} = 0$, and the tangent to that conic at a point $\mathbf{p}$ on it is $\mathbf{l} = \mathbf{C}\mathbf{p}$. Both tasks are those four formulas run on measurements you make yourself.

Task 1	Vanishing points and the vanishing line	60 pts
Task 2	A circle photographed from an angle	40 pts
	Total	100 pts
Bonus	Shooting around cover (optional)	+10 pts

A skeleton, `hw1_skeleton.py`, comes with the assignment. The framework is written for you: reading the images, drawing the overlays, saving the figures. Every place that needs your work is marked `TODO`.

1.1 Quick reference: where each task comes from

Topic	Lecture 2 pages	Used in
Homogeneous coordinates for points and lines	2-1 to 2-2	everywhere
Meet and join as cross products	2-2	Task 1, Bonus
Ideal points and $\mathbf{l}_\infty$	2-2 to 2-3	Task 1, Bonus (a) and (d)
The vanishing line	2-3	Task 1
Conics, $\mathbf{x}^T\mathbf{C}\mathbf{x} = 0$	2-3 to 2-4	Task 2
The tangent $\mathbf{l} = \mathbf{C}\mathbf{p}$	2-4	Task 2(d)

2 Notation

Boldface lowercase letters denote vectors, boldface uppercase letters denote matrices, and lowercase letters without a special typeface denote scalars. A point (x, y) is written in homogeneous coordinates as $[x, y, 1]^T$, and two homogeneous quantities equal up to a nonzero scale factor are written with $\equiv$.

A conic is written in the implicit form of Lecture 2,

$$ax^2 + bxy + cy^2 + dx + ey + f = 0, \qquad \mathbf{C} = \begin{bmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & f \end{bmatrix},$$

so that $a, \ldots, f$ always name these six coefficients.

The bonus section is set in an ordinary x-y plane with y upward and α measured counter-clockwise from the $+x$ axis. Both graded tasks work directly in pixel coordinates, which are (column, row) with row 0 at the top of the image.

```
import numpy as np
join = lambda p, q: np.cross([p[0], p[1], 1.], [q[0], q[1], 1.])  # line
    thru 2 points
meet = lambda l, m: np.cross(l, m)                                  #
   where 2 lines cross

def dehom(x, eps=1e-9):
    # A homogeneous vector is defined only up to scale, so "x3 is small"
        means
    # nothing on its own: compare it against |x1| and |x2|.  Note that
       [0,0,c]
    # with c nonzero is the ORIGIN, not an ideal point, and must
       dehomogenize.
    if not np.any(x): return None                    # the zero vector:
       not a point
    s = max(abs(x[0]), abs(x[1]))
    return None if abs(x[2]) <= eps * s else x[:2] / x[2]        # None
       = ideal point
```

3 Setting up your environment

3.1 Python environment

Use Anaconda, so that your environment is reproducible and we can run your code. If you already have `conda` working, skip to the second box.

Installing it. Go to https://www.anaconda.com/download and take either installer for your operating system. Both are free and both give you the same `conda` command:

- **Miniconda** is `conda` plus Python and nothing else, about 900 MB installed. It is the one to take. Everything this homework needs, you install yourself in the next step.
- **Anaconda Distribution** is the same thing with several hundred packages preinstalled, about 9.7 GB. Take it only if you want the extras.

The download page asks you to register. Use your Purdue address. Anaconda's academic policy covers coursework at accredited universities at no cost.

Accept the installer's defaults, and when it offers to initialize your shell, let it. Then close your terminal and open a new one, because the initialization only takes effect in a shell started afterwards. On Windows, open Anaconda Prompt from the Start menu rather than PowerShell or `cmd`; on macOS and Linux any terminal will do. Check that it worked:

```
conda --version            # prints a version number if all is well
```

If instead you get `conda: command not found`, your shell was not initialized. Reopen the terminal first. If it still fails, run the initializer by hand: `~/miniconda3/bin/conda init` on macOS and Linux, or reinstall on Windows with the "Add to PATH" box ticked.

Building the environment for this course. One line makes it, one line enters it, and the third writes the file you submit:

```
conda create -n ece661 python=3.11 numpy matplotlib pillow
conda activate ece661
conda env export > environment.yml      # submit this with your code
```

The prompt should now start with `(ece661)`. You have to run `conda activate ece661` in every new terminal; it does not stick. Run your code from inside that environment, and regenerate `environment.yml` at the end, after you have installed everything you ended up using.

Do not use OpenCV in this homework. NumPy, Matplotlib and Pillow are enough.

3.2 Reading pixel coordinates off an image

Both graded tasks ask you to read the pixel coordinates of features you choose. Any tool that reports the cursor position will do. IrfanView, Paint, ImageMagick's `display` and Matplotlib's interactive window all work. GIMP is free, behaves the same on Windows, macOS and Linux, and is what the rest of this section assumes. Say in your report which tool you used.

1. Install GIMP from https://www.gimp.org/downloads/. Open your image with File → Open.
2. Open Windows → Dockable Dialogs → Pointer. A small panel appears; its Pixels row is the readout you want. Use it rather than the figure in the bottom-left corner of the window, which is easy to misread.
3. Check the units. The dropdown beside the zoom box at the bottom of the image window should read px. If it says in or mm every number you record will be wrong.
4. Zoom in before you record anything. Press + and −, or type a percentage into the zoom box. Work at 200% to 400%: at 100% you cannot place a point to better than two or three pixels, and that error propagates into everything downstream. Hold the space bar and drag to pan.
5. Move the pointer onto the feature and read Pixels x and y from the Pointer panel. You do not need to click at all. Hovering updates the readout, and not clicking means you cannot accidentally paint on the image.
6. **Check your setup before you record eighteen points.** Open `plate.jpg` and put the pointer on the leftmost point of the plate's outer edge. You should read approximately (485, 675). If you do not, something above is wrong. The usual causes are the units, or reading the status bar instead of the Pointer panel. It is much cheaper to find that out now than after eighteen points.
7. Record as you go, straight into the lists in `hw1_skeleton.py`. Tabulate them in your report as well; you are marked on those tables.

4 Task 1: Vanishing points and the vanishing line (60 pts)

Lecture 2 states that every ideal point lies on $\mathbf{l}_\infty = [0, 0, 1]^T$, and that the camera image of $\mathbf{l}_\infty$ is called the Vanishing Line. This task measures it. In an image the ideal point of a family of parallel lines appears as an ordinary finite point, its *vanishing point*, and the claim becomes testable: the vanishing points of different families of parallel lines lying in one plane of the world should be collinear. Alberti wrote down in 1435 the rule that lines running back into a scene converge to one point on a horizontal line at eye level. What he did not have is the general statement, that this holds for *every* family of parallel lines in the plane whatever direction it runs in. That is what you are measuring.

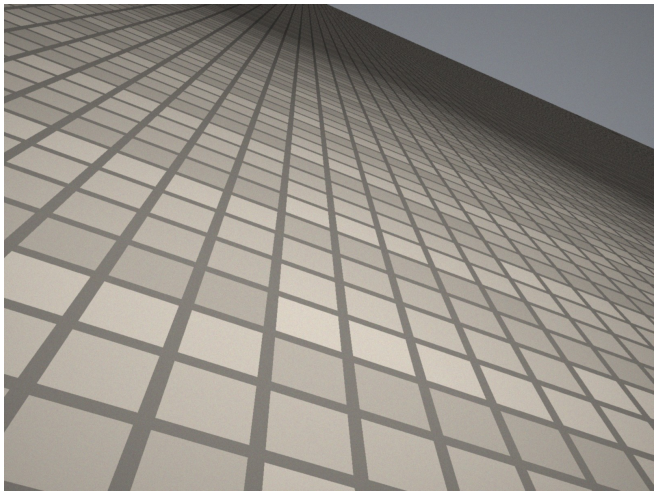


Figure 1: `tiles.jpg`, 1600×1200. This image and `plate.jpg` are computer renderings, not photographs. They are rendered rather than shot so that their true geometry is known exactly to the graders, who mark against it. Part (e) sends you to a real camera, where your numbers will be worse, and Task 1(e) asks you why.

(a) Work from `tiles.jpg` (Figure 1). Pick three families of parallel lines lying in the plane of the plaza: the two tile directions, and the tile diagonals. The diagonals are not drawn in the image; you locate one by recording a corner where four tiles meet and then stepping diagonally across the grid to another such corner. Count the tiles carefully, because a miscount gives you a line that is not in the family, and nothing in the image will tell you so. Plotting your recorded points on the image before you compute anything will. Do that first; it takes two seconds and it is the only check you get on this step. In each family take three lines and record two well-separated points on each, eighteen points in all. Tabulate them. **(10 pts)**

(b) For each family, build its three lines as $\mathbf{l} = \mathbf{x}_1 \times \mathbf{x}_2$ and compute the vanishing point as $\mathbf{v} = \mathbf{l}_i \times \mathbf{l}_j$ for all three pairs. Report the three estimates and their *spread*, the diagonal of their bounding box.

Then dehomogenize each of the three estimates and take that family's vanishing point to be the **arithmetic mean of their Cartesian** (x, y) **coordinates**. Dehomogenize first and average second: a homogeneous 3-vector is fixed only up to scale, so averaging the 3-vectors themselves is not a defined operation and would give a different answer for every choice of scale. Use that single point everywhere from here on. Averaging the three is both a better estimate than any

one of them and, more to the point here, a definite one: parts (c) and (d) would otherwise give a different answer depending on which pair of lines you happened to pick, and on this image that answer moves by a factor of several hundred. Report each family's mean vanishing point and its distance from the centre of the image, which for a 1600×1200 image is $(800, 600)$.

Before you compute any of this, write down a prediction: which of your three families will give the least reliable vanishing point, and why? Then compute all three and say whether the prediction held. If it did not, say what you think happened.

(17 pts)

(c) Order your three families by spread. Three points are collinear exactly when the third lies on the line through the other two, so test collinearity directly: report the perpendicular distance in pixels from the *worst-determined* vanishing point to the line through the other two. (Three homogeneous points are collinear exactly when $\det[\mathbf{v}_1\ \mathbf{v}_2\ \mathbf{v}_3] = 0$; you are asked for the distance rather than the determinant, and part of this question is why. Each $\mathbf{v}_i$ is fixed only up to scale, so multiplying one of them by 10^6 multiplies the determinant by 10^6 without moving a single point. Work out what that does to the determinant as a measure of anything.) Say what your distance measures, and why it is a more useful thing to report.

Then close the loop on the claim in the preamble: in one sentence, what does your number say about it, and what result would have falsified it? **(12 pts)**

(d) Compute the vanishing line $\mathbf{l}_v$ as the join of the two *best* determined vanishing points, and plot the image, your recorded points, the nine lines extended, the vanishing points and $\mathbf{l}_v$. On this image no single set of axis limits shows both the picture and the most distant vanishing point usefully, so use two panels: a near view holding the image and the two nearer vanishing points, and a wide view holding all three. Then name the case that a Cartesian implementation would have to special-case and that the cross product handles with no special case at all. It is not the vanishing point being far away, which is an ordinary pair of numbers; it is the limiting case just beyond that. Say where a test is still needed, and what that tells you about where the difficulty actually lives. **(9 pts)**

(e) Photograph a scene of your own containing at least three families of parallel lines lying in one plane of the world: a tiled floor, a brick wall, a parking lot, a chessboard on a table. Shoot it obliquely so the lines visibly converge. Repeat (a) to (d) on it. On your own photograph one or more of the vanishing points may well land inside the frame; if so, say so and use a single panel.

Then compare your collinearity residual with the one you got on `tiles.jpg`. It will almost certainly be worse. Give a lens-related reason, and say how you would tell that cause apart from simple carelessness in recording. Take the photograph on the first day you start this homework, not the last. **(12 pts)**

5 Task 2: A circle photographed from an angle (40 pts)

(a) Work from `plate.jpg` (Figure 2), a circular plate seen obliquely. Record eight points spread right around the *outer* edge of the plate, where it meets the table. Do not mix in points from the inner ring where the glaze changes shade: that is a second, smaller circle, and points from both will not lie on any one conic. Tabulate them. **(8 pts)**

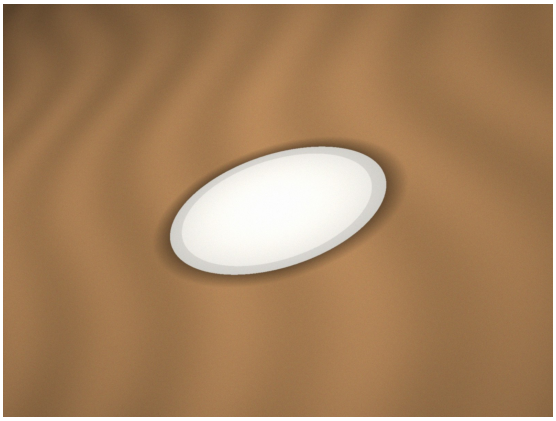


Figure 2: `plate.jpg`, 1600×1200. A rendering; see the caption of Figure 1.

(b) Each point on a conic contributes one row $[\,x^2,\ xy,\ y^2,\ x,\ y,\ 1\,]$ to a design matrix $\mathbf{A}$, and since $\mathbf{C}$ is fixed only up to scale, the coefficient vector is the right singular vector belonging to the smallest singular value of $\mathbf{A}$, which is the last *row* of `Vt` if you are using `numpy.linalg.svd`. Fit $\mathbf{C}$ this way and report it.

Keep the shape of this construction in mind. Building a design matrix whose null vector is the answer, because the answer is fixed only up to scale, is exactly how you will estimate a homography in HW2.

Then scale $\mathbf{C}$ so that $\|\mathbf{C}\|_F = 1$ and report $\max_i |\mathbf{x}_i^T \mathbf{C} \mathbf{x}_i|$ over your eight points. Fixing the scale is not optional: $\mathbf{x}^T\mathbf{C}\mathbf{x}$ is linear in $\mathbf{C}$, so without a stated convention that number can be made anything at all. Expect something of order 10^{-3} to 10^{-4}; if it comes out of order 1, you have taken the wrong singular vector. Say what the number measures and why it is not a distance in pixels. **(13 pts)**

(c) Rescale the coefficient vector so that $a^2 + b^2 + c^2 = 1$, report the discriminant $b^2 - 4ac$ and classify the conic. For a nondegenerate real conic, negative is an ellipse, positive a hyperbola, and zero a parabola; a fit from eight recorded points is nondegenerate unless something has gone badly wrong, and the checks below will tell you if it has. Then answer this. The object is a *circle* in the world. In a Euclidean Cartesian frame a nondegenerate conic is a circle exactly when its coefficients satisfy $b = 0$ and $a = c$, which is two constraints on the five degrees of freedom a general conic has. Note that this is a statement about one particular frame, not a property the curve carries around with it. Your fitted $\mathbf{C}$ satisfies neither constraint. Give a reason why a camera in a general position should not be expected to preserve relations between individual entries of $\mathbf{C}$, and say which properties of the shape did survive.

Three numbers to check yourself against before you go further. On `plate.jpg` the discriminant should land near -0.86. If you get close to -2, you have fitted under a circle constraint rather than a general conic. If you get something near zero, you have normalized all six coefficients instead of just a, b and c. If you get a positive number your fit has failed, and the usual cause is points bunched on one short arc of the rim. **(11 pts)**

(d) The matrix $\mathbf{C}$ of Section 2 is symmetric by construction, since the xy, x and y coefficients are split evenly across the off-diagonal pairs. You will need that fact twice below. Pick one of your recorded points $\mathbf{p}$ and compute the tangent there as $\mathbf{l} \equiv \mathbf{C}\mathbf{p}$, the $\equiv$ being the one from Section 2: $\mathbf{l}$ is a homogeneous line and any nonzero multiple of $\mathbf{C}\mathbf{p}$ names it equally

well. Overlay the fitted conic, the eight points and the tangent on the image, then zoom in on $\mathbf{p}$ and confirm that the line touches the curve there without crossing it. That, and not the arithmetic below, is the check that your $\mathbf{C}$ is right.

Then compute $\mathbf{p}^T\mathbf{C}\mathbf{p}$ and $\mathbf{l}^T\mathbf{p}$. They will agree. Say why they must, and say why their agreeing tells you nothing about whether your fit is any good. **(8 pts)**

6 Bonus: Shooting around cover (10 bonus pts)

This section is optional and worth 10 bonus points on top of the 100. It needs no images and no GIMP, only the four formulas of Lecture 2 in an ordinary x-y plane. Attempt it after Tasks 1 and 2 are finished.

A line in homogeneous coordinates has no direction. Everything here is built on that one fact, in the setting of an arcade game where shells ricochet off the arena walls and the only way to score is to aim at a wall rather than at the target.

Figure 3 shows the arena. The player sits at the origin. A target bar runs from $(1, 8)$ to $(3, 8)$, armour runs from $(0.4, 4)$ to $(5, 4)$, and a mirror wall stands at $x = 7$ over $0 \le y \le 6$.

(a) Write the aiming line for angle α in homogeneous coordinates and say where α appears in it. Report the angular intervals subtended at the origin by the target and by the armour. Containment of one interval in the other is not by itself enough to conclude that the target is unreachable, so say also why the armour is met *first*, and conclude that no direct shot exists.

Then meet your aiming line with $\mathbf{l}_\infty = [0, 0, 1]^T$ and say what you get. Be warned that $[0, 0, 1]^T$ names two different things on this page: read as a point it is the origin, read as a line it is $\mathbf{l}_\infty$. That is duality and not a typo. Keep the answer, because part (d) uses it. **(2 pts)**

(b) Implement the obvious hit test: build the line through each segment's endpoints, meet it with the aiming line, and count a hit whenever that meet falls between the two endpoints. Run it at $\alpha = 250°$, which points away from everything, and report what it says about the target. Look again at where α ended up in your answer to (a); that is the whole diagnosis. State the missing condition. It is a *ray-direction test*, not an adjustment to the angle bookkeeping. With $\mathbf{d} = (\cos\alpha, \sin\alpha)^T$ and $\mathbf{q}$ the meet in Cartesian coordinates, write it as the inequality $\mathbf{d}^T\mathbf{q} > 0$ and say why the inequality is strict. Fix the code and rerun. **(4 pts)**

(c) A reflection in the line $\mathbf{l} = [a, b, c]^T$ with $a^2 + b^2 = 1$ is $\mathbf{M} = \mathbf{I} - 2\tilde{\mathbf{n}}\mathbf{l}^T$ with $\tilde{\mathbf{n}} = [a, b, 0]^T$, because $ax + by + c$ is the signed distance to the line and (a, b) is its unit normal. Write $\mathbf{M}$ out for the wall $x = 7$, verify that $\mathbf{M}^2 = \mathbf{I}$, and say in one sentence why that had to be true. **(1 pt)**

(d) Reflect the target bar in the wall, aim at the mirror image, and compute the angle that reaches the centre of the bar. Verify it by tracing the ray yourself: confirm that it clears the armour, find where it strikes the wall, reflect the direction, and check that the return leg lands on the bar.

To reflect the direction, apply $\mathbf{M}$ to the ideal point you computed in part (a), $[\cos\alpha, \sin\alpha, 0]^T$. The translation column of $\mathbf{M}$ multiplies the third coordinate, which is zero, so it drops out and what comes back is the reflected direction. A direction *is* an ideal point, which is why

one matrix serves for both jobs and why part (a) was worth doing. Report every intermediate number and plot the two-leg path. **(3 pts)**

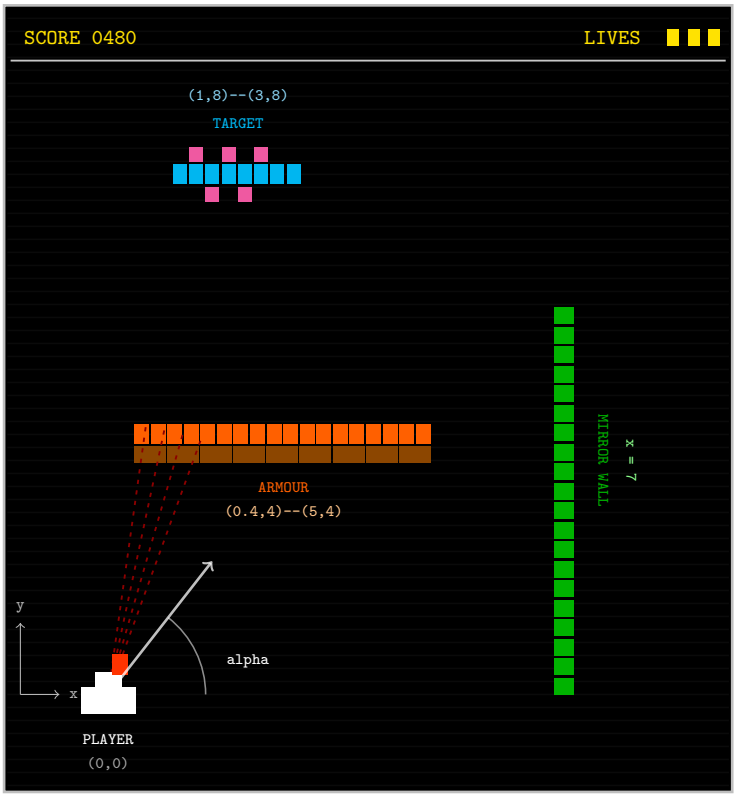


Figure 3: The arena. The dashed rays from the player are four directions you might try; part (a) asks what becomes of them.

7 Submission instructions

Read this section carefully. Incomplete submissions will lose points. Every item listed below must be present in your submission.

Everything goes through **Gradescope**, in **two separate submissions**. Submit to both.

7.1 What your report must begin with

Two things come before Task 1, in this order.

1. **The signed policy declaration.** The course policy sheet, filled in and signed. Make sure you submit this at the right submission link before you begin writing your report.
2. **A section titled "AI Assistant Usage".** Course policy requires this at the *start* of every submission, whether or not you used one. State how much of your solution was generated by an AI assistant, as a percentage, in the form `AI Usage: XX%`. If you used none, say `AI Usage: 0%`. Any function in your code that was generated entirely by an AI assistant must say so in its

docstring. Read the policy sheet for what happens if work is found to be plagiarised; it is not a formality.

7.2 What to submit for each task

1. **Task 1: Vanishing points and the vanishing line (60 points)**
   - Code: your implementation
   - Table: the eighteen recorded coordinates, and the tool you used
   - Text: your written prediction, made *before* you computed anything
   - Table: three vanishing point estimates per family, their spreads, the mean vanishing point of each family, and its distance from $(800, 600)$
   - Text: whether the prediction held, and the account of it
   - Text: the collinearity residual in pixels, what it measures, why it is reported instead of the determinant, and what it says about the claim
   - Figure: the two-panel plot with the nine lines, the vanishing points and $\mathbf{l}_v$
   - Text: the degenerate case, and where a test is still needed
   - Media: your own photograph for part (e)
   - Table and Figure: the same quantities repeated on your own photograph
   - Text: the comparison of residuals and the lens-related reason
2. **Task 2: A circle photographed from an angle (40 points)**
   - Code: your implementation
   - Table: the eight recorded rim coordinates
   - Text: the fitted $\mathbf{C}$, and $\max_i |\mathbf{x}_i^T \mathbf{C} \mathbf{x}_i|$ at $\|\mathbf{C}\|_F = 1$, with what that number measures and why it is not a distance
   - Text: the discriminant, the classification, and why neither constraint survived
   - Figure: the fitted conic, the eight points and the tangent overlaid, with a zoomed view showing the tangency
   - Text: why $\mathbf{p}^T \mathbf{C} \mathbf{p}$ and $\mathbf{l}^T \mathbf{p}$ agree, and why that verifies nothing
3. **Bonus: Shooting around cover (10 bonus points, optional)**
   - Code: the aiming line, the hit test and the bank shot
   - Text: the two angular intervals, why the armour is met first, and the $\mathbf{l}_\infty$ meet
   - Text: what the naive test reports at $\alpha = 250°$, and the corrected condition with the strictness argued
   - Text: $\mathbf{M}$ for $x = 7$, and the check that $\mathbf{M}^2 = \mathbf{I}$
   - Text: the aiming angle and every intermediate number of the two-leg trace
   - Figure: the two-leg path plotted on the arena

7.3 Code files to submit

1. `hw1_<FirstName><LastName>.py`
2. `environment.yml`, exported with `conda env export`

3. `README.md`: how to run your code, any seeds you used, and your compute setup

You do not submit `tiles.jpg` or `plate.jpg`. They are provided data, not your work, and we already have them.

7.4 Interface requirements

Your submitted `.py` file must be safe to *import*. Keep the work inside functions and keep the calls behind `if __name__ == '__main__':`, exactly as `hw1_skeleton.py` does.

7.5 Code quality requirements

- Well-commented code explaining what each section does
- Meaningful variable and function names
- Docstrings for all functions
- Code should run without modification, given the environment of Section 3
- Follow PEP 8 style guidelines

7.6 Report formatting

- **Include code snippets in the report for every task.** For each task, show the code you wrote alongside the outputs it produced. If the code snippet is not present, you will lose points on that task.
- Place output directly next to the corresponding code block.
- Use figures and tables with captions.
- Number equations, figures and tables.
- Use proper citations when referencing papers, code repositories or course materials.

7.7 Submitting on Gradescope

1. `HW1 Report` takes one PDF.

1. Upload a PDF, not a zip. This assignment type accepts PDFs only.
2. Start each lettered part on a new page. "Part" means a lettered part of a task statement: Task 1(a), Task 1(b), and so on, nine in all, plus four more if you attempt the bonus. The AI Assistant Usage section is tagged too. You will be asked to tag which pages hold which part.
3. **Tag every part.** An untagged part is a part the grader never sees, and it is marked zero. Gradescope will not warn you. This is the single most common way to lose points on a correct submission.
4. Keep the file under 100 MB. Export figures as JPEG or as PNG at a sensible resolution.
5. You may resubmit before the deadline; the last submission is graded, and your tags carry over.

2. `HW1 Code` takes the files listed in Section 7.3, either individually or as a single zip.

7.8 Additional guidelines

- Submit `.py` files. Do **not** submit `.ipynb`. If you worked in a notebook, convert it to a script before submitting.
- If submitting late, submit only once.
- This is individual work. Your code and your report must be your own.
- AI assistant usage is declared at the *front* of the report, as Section 7.1 sets out, not in a note at the end.
- Your numbers will not match a classmate's, because you recorded different pixels. Report your own.

8 Frequently asked questions (FAQ)

Q: GIMP reads coordinates that do not match the check in Section 3.2.

A: Look at the dropdown beside the zoom box at the bottom of the image window. If it reads in or mm instead of px, every number you record will be wrong. If the units are right, you are probably reading the figure in the bottom-left corner of the window rather than the Pointer panel.

Q: All three of my vanishing points are far outside the image.

A: Expected on `tiles.jpg`, and Task 1(d) asks you to plot them anyway. One of them will be far enough out that a single set of axis limits cannot show both it and the picture, which is why that part asks for two panels.

Q: My three families give spreads in a different order from the one I predicted.

A: That happens, and it is not a mistake. Recording the same nine lines a second time moves the worst family's spread over more than a tenfold range. Task 1(b) is marked on the prediction and the reasoning behind it, not on which family won.

Q: One line-pair in a family disagrees with the other two by thousands of pixels.

A: You miscounted tiles when stepping along a diagonal, so one of your three lines is not in that family. Plot your recorded points on the image and you will see it.

Q: My fitted conic does not follow the rim of the plate.

A: Either your eight points are bunched along one short arc, or you mixed points from the outer edge of the plate with points from the inner ring, which is a different circle. Spread the points right around one boundary.

Q: My discriminant is nowhere near -0.86.

A: Near -2 means you fitted under a circle constraint instead of a general conic. Near zero means you normalized all six coefficients instead of just a, b and c. Positive means the fit failed, and the usual cause is the previous question.

Q: `numpy.linalg.svd` gives me a conic that plots as nothing.

A: You have taken the wrong singular vector. You want the last *row* of `Vt`, not a column of `V` and not anything from `U`. Check the residual: at $\|\mathbf{C}\|_F = 1$ it should come out around 10^{-3} to 10^{-4}, not order 1.

Q: `np.cross` warns about 2-element vectors.

A: Stay in three-element homogeneous vectors throughout.

Q: My numbers do not match a classmate's.

A: They will not. You recorded different pixels. Report your own.

Q: Can I work in a group?

A: No. This is individual work.

Q: I did not use an AI assistant. Do I still need the AI Assistant Usage section?

A: Yes. Write `AI Usage: 0%`. The section is required on every submission; its absence is what gets noticed, not its contents.

Q: How exact does the AI percentage have to be?

A: An honest estimate of the fraction of your submitted solution that came from an assistant, to the nearest per cent, is what is being asked for.